

Ing. Sistemas Computacionales

9°A

Configuración de protocolos

HTTP / HTTPS

Y despliegue WEB

Marco Antonio Elias Lozano

9079

Escolarizada

Fecha: 22/06/2026

Fundamentos y Funcionamiento de HTTP/HTTPS

El **Protocolo de Transferencia de Hipertexto (HTTP, por sus siglas en inglés)** es el protocolo base utilizado en la World Wide Web para la transmisión de datos y páginas HTML entre un servidor web y un navegador cliente. Opera de manera nativa sobre el modelo TCP/IP utilizando el **puerto 80**.

Por otro lado, **HTTPS** representa la versión segura del protocolo, incorporando una capa de cifrado mediante certificados SSL/TLS que viaja a través del **puerto 443**. Esta división de puertos garantiza que el servidor pueda identificar si una solicitud requiere una conexión abierta estándar o un canal cifrado de alta seguridad para proteger la integridad y privacidad de la información transferida.

Limpieza del Directorio Raíz y Gestión de Archivos en el Servidor

Utilizando el cliente SSH Termius, se accedió a la ruta por defecto del servidor web de Nginx en `/var/www/html`. Tras auditar el directorio, se detectó un archivo `index` previo. Para realizar un despliegue limpio, se removió de forma segura la estructura antigua mediante el comando:

```
sudo rm -f /var/www/html/index.html
```

Posteriormente, utilizando el editor de texto `nano`, se creó un nuevo archivo `index.html` en la misma ruta raíz para albergar el código fuente del sitio web del proyecto.

```
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 3
root@ip-172-31-9-49:~# cd /var/www/html
root@ip-172-31-9-49:/var/www/html# ll
total 8
drwxr-xr-x 2 root root 4096 Jun 23 15:26 ./
drwxr-xr-x 3 root root 4096 Jun 23 14:52 ../
root@ip-172-31-9-49:/var/www/html# nano index.html
root@ip-172-31-9-49:/var/www/html# sudo cp index.html /var/www/html/
cp: 'index.html' and '/var/www/html/index.html' are the same file
root@ip-172-31-9-49:/var/www/html# sudo systemctl status apache2
Unit apache2.service could not be found.
root@ip-172-31-9-49:/var/www/html# sudo nano index.html
root@ip-172-31-9-49:/var/www/html# pwd
/var/www/html
root@ip-172-31-9-49:/var/www/html# ls -la
total 12
drwxr-xr-x 2 root root 4096 Jun 23 16:26 .
drwxr-xr-x 3 root root 4096 Jun 23 14:52 ..
-rw-r--r-- 1 root root 330 Jun 23 16:20 index.html
root@ip-172-31-9-49:/var/www/html# cat /var/www/html/index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gist Embebido</title>
</head>
<body>

  <h1>Contenido del Gist</h1>

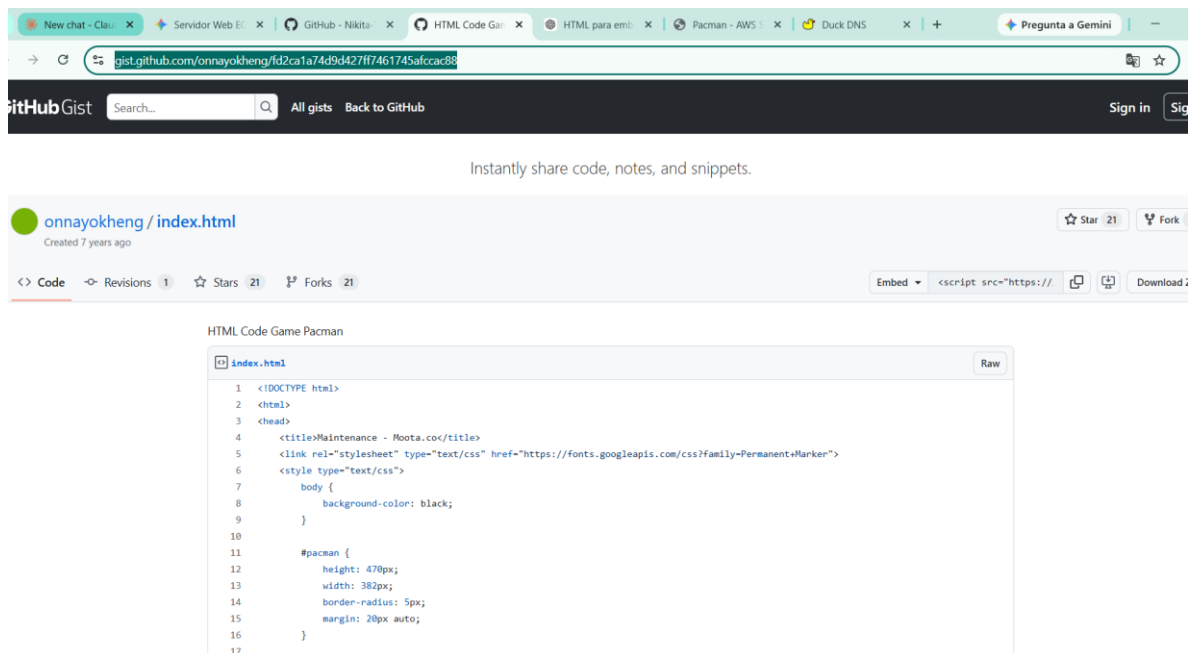
  <script src="https://gist.github.com/onnayokheng/f2ca1a74d9d427ff7461745afccac88.js"></script>

</body>
</html>
root@ip-172-31-9-49:/var/www/html# sudo rm -f /var/www/html/index.html
root@ip-172-31-9-49:/var/www/html# sudo nano /var/www/html/index.html
root@ip-172-31-9-49:/var/www/html# sudo nano /var/www/html/index.html
root@ip-172-31-9-49:/var/www/html# sudo nano /var/www/html/index.html
root@ip-172-31-9-49:/var/www/html#
```

Integración de Código Fuente Externo

(GitHub Gist)

Para el desarrollo de la interfaz de la página (en este caso, un entorno interactivo del juego Pacman), se localizó un repositorio de código fuente público alojado en GitHub Gist (onnayokheng / index.html). Se analizó la estructura del código HTML5 base, identificando las clases de estilos CSS y los scripts necesarios para su correcto renderizado en el navegador web.



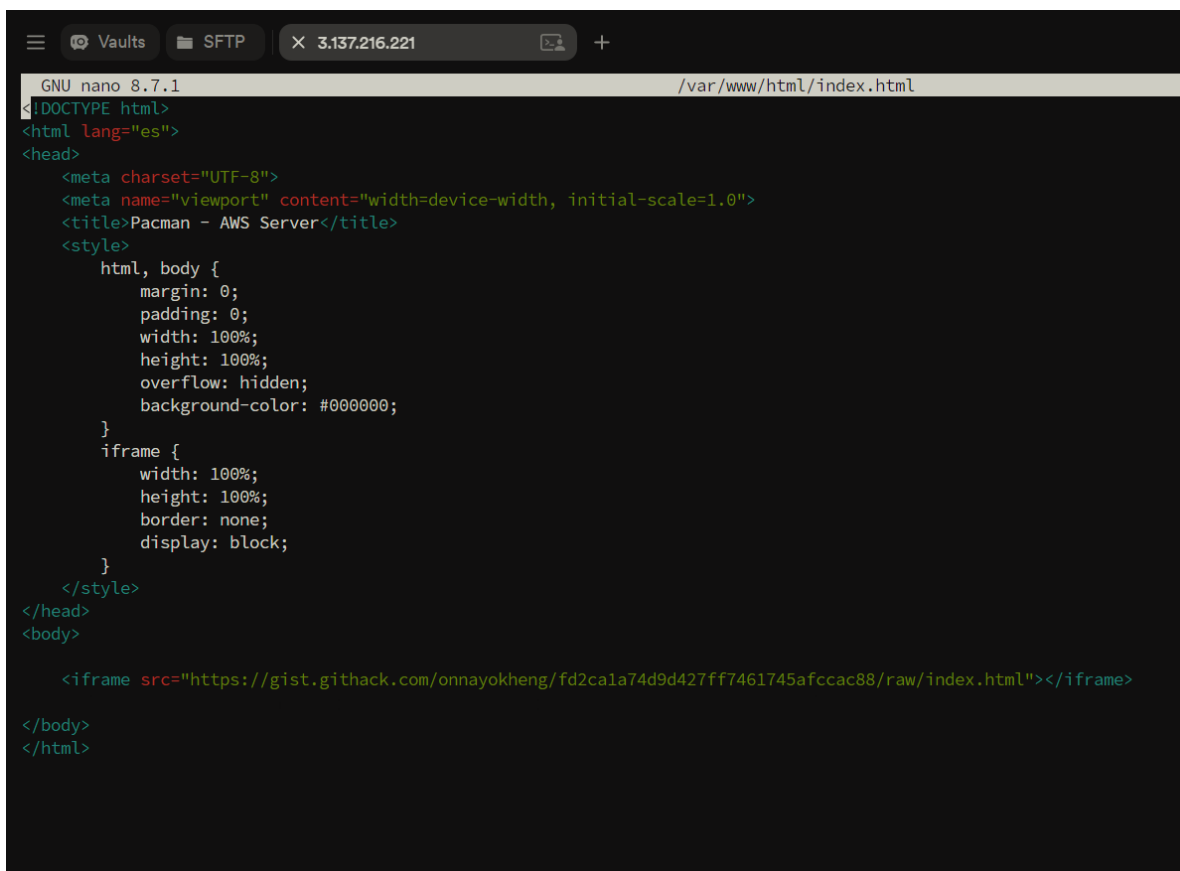
The screenshot shows a web browser with multiple tabs open. The active tab is a GitHub Gist page for the file 'index.html' by user 'onnayokheng'. The page title is 'HTML Code Game Pacman'. The code is displayed in a light blue editor with line numbers 1 through 17. The code includes HTML5 boilerplate, a title 'Maintenance - Mooto.co', a link to a Google Fonts stylesheet, and CSS rules for the body and a Pacman character.

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Maintenance - Mooto.co</title>
5   <link rel="stylesheet" type="text/css" href="https://fonts.googleapis.com/css?family=Permanent+Marker">
6   <style type="text/css">
7     body {
8       background-color: black;
9     }
10
11     #pacman {
12       height: 470px;
13       width: 382px;
14       border-radius: 5px;
15       margin: 20px auto;
16     }
17
```

Embebido y Maquetación mediante iFrame en Nginx

De vuelta en el servidor mediante el editor nano, se programó el archivo `/var/www/html/index.html`. Con la finalidad de optimizar el consumo de almacenamiento local de la instancia EC2, se optó por una arquitectura de integración híbrida utilizando una etiqueta `<iframe>` configurada a pantalla completa (`width: 100%; height: 100%;`), mandando llamar de forma directa el recurso crudo del proyecto desde el servidor de desarrollo (`gist.githack.com`).

```
<iframe  
src="https://gist.githack.com/onnayokheng/fd2ca1a74d9d427ff7461745afccac  
88/raw/index.html"></iframe>
```



```
GNU nano 8.7.1 /var/www/html/index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pacman - AWS Server</title>
  <style>
    html, body {
      margin: 0;
      padding: 0;
      width: 100%;
      height: 100%;
      overflow: hidden;
      background-color: #000000;
    }
    iframe {
      width: 100%;
      height: 100%;
      border: none;
      display: block;
    }
  </style>
</head>
<body>

  <iframe src="https://gist.githack.com/onnayokheng/fd2ca1a74d9d427ff7461745afccac88/raw/index.html"></iframe>

</body>
</html>
```

Configuración de Reglas de Seguridad Perimetral

(Puertos 80 y 443)

Para que las peticiones HTTP y HTTPS externas procedentes de cualquier parte del mundo puedan acceder al servidor Nginx, se validó la matriz de cortafuegos perimetral del **Security Group** asignado a la instancia EC2 de AWS.

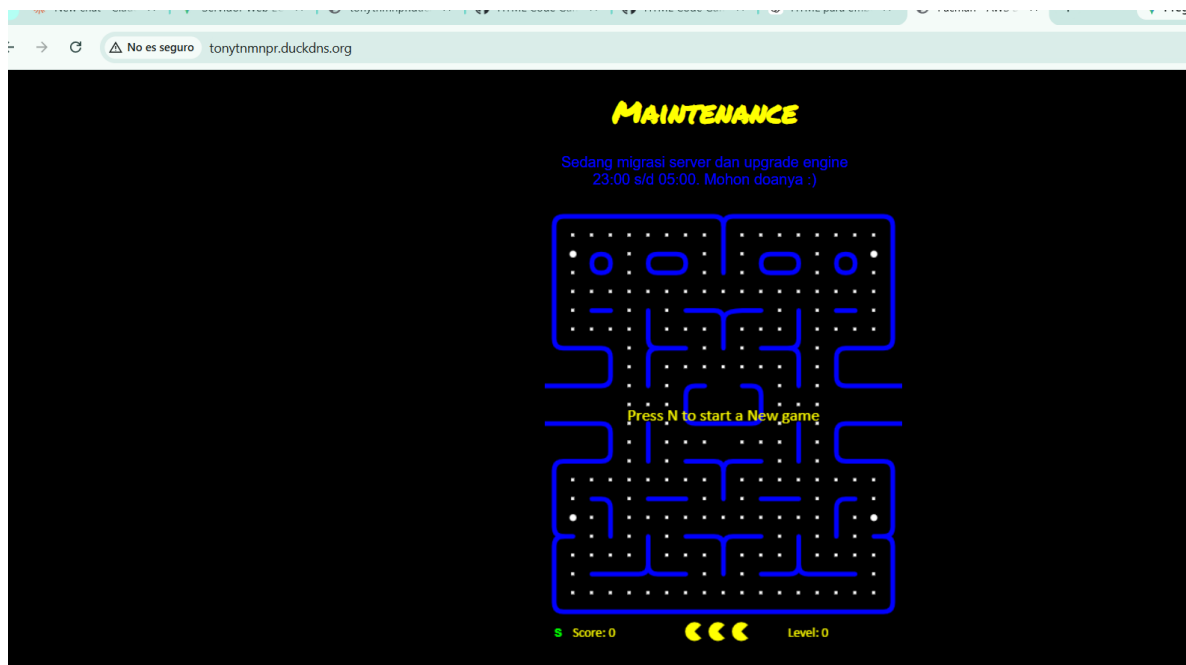
Aunque en la consola se administran puertos de gestión avanzada (como DNS 53, RDP 3389 y NTP 123), se verificó la correcta habilitación de las reglas de ingreso esenciales para la web:

- **HTTP (Puerto 80):** Tráfico web entrante estándar para la carga inicial de la página.
- **HTTPS (Puerto 443):** Preparación del canal para la posterior capa de cifrado y seguridad del dominio.

Información					
sgr-07c34b0c7c366fd3d	DNS (TCP)	TCP	53	Persona...	Q
sgr-020d9fc3b762eb2c8	DNS (UDP)	UDP	53	Persona...	Q
sgr-05b7d8c40d360b8b7	HTTPS	TCP	443	Persona...	Q
sgr-0c3ebfff29c3d36b	TCP personalizado	TCP	23	Persona...	Q
sgr-084fc39cb14c4fc8	RDP	TCP	3389	Persona...	Q
sgr-0069437dfd7661ac9	HTTP	TCP	80	Persona...	Q

Validación Final del Despliegue en Producción

Una vez guardados los cambios en Nginx y comprobada la apertura de puertos, se procedió a testear la conectividad en tiempo real. Al introducir el nombre de dominio enlazado en la práctica anterior (tonytnmnpr.duckdns.org), el navegador web resolvió de forma inmediata el puerto 80 del servidor web en la nube, cargando y renderizando correctamente la interfaz del juego interactivo sin retrasos ni errores de denegación de red.



¿Para qué se realizó y cómo nos funciona?

- **¿Para qué?:** El mapeo y la apertura de los puertos HTTP (80) y HTTPS (443) se realizó con el objetivo de **democratizar el acceso a la instancia**. Por defecto, una instancia en la nube bloquea todo el tráfico entrante por seguridad. Habilitar explícitamente estos protocolos permite que el servidor actúe formalmente como un servidor web público capaz de escuchar y responder peticiones de navegación de usuarios externos.
- **¿Cómo funciona?:** Cuando un cliente escribe el dominio en su barra de navegación, el explorador web genera una solicitud HTTP por defecto dirigida al **puerto 80** de la IP del servidor. El grupo de seguridad de AWS analiza la regla, comprueba que el tráfico en ese puerto está permitido y lo deja pasar. Nginx toma la petición, busca el archivo index.html que acabamos de editar y le responde al cliente enviándole las etiquetas del iframe. Finalmente, el navegador procesa el código y muestra la página web de forma interactiva.

Conclusión

La realización exitosa de esta práctica confirma que el despliegue de cualquier aplicativo o sitio web en la nube requiere una sincronización exacta entre la programación interna del sistema operativo y las políticas de red externas de la infraestructura.

La correcta manipulación de archivos web mediante SSH (Termius), en conjunto con una maquetación limpia basada en recursos externos (GitHub) y la apertura estricta de los puertos perimetrales HTTP y HTTPS en AWS, consolida las capacidades prácticas indispensables para publicar proyectos web interactivos bajo un entorno escalable, eficiente en el uso de recursos y accesible de forma universal a través de la red global.